

# 데이터 시각화

# 데이터를 시각화하는 방법

---

## ● 파이썬 기반의 시각화 라이브러리

- seaborn
  - matplotlib을 기반으로 다양한 색 테마와 차트 기능을 추가한 고수준(high-level) 인터페이스
  - 즉, matplotlib에 비해 사용하기 쉬운 패키지
  - 그래프를 훨씬 수월하게 그릴 수 있을 뿐만 아니라 시각적으로 다양한 효과를 낼 수 있음
  - 여기서는 그래프를 그리는 물감으로 활용
- matplotlib
  - 데이터 시각화, 즉 다양한 그래프를 그리는 라이브러리를 제공하는 패키지
- plotly
  - 인터랙티브 웹 기반 시각화의 선두주자
  - 정적인 이미지가 아닌 사용자가 직접 조작할 수 있는 살아있는 그래프를 만들 수 있어, 프레젠테이션이나 대시보드 구축에 탁월
  - 특히 비전공자들과의 소통이 중요한 비즈니스 환경에서 그 진가를 발휘

# seaborn

---

- matplotlib을 기반으로 다양한 색 테마와 차트 기능을 추가한 고수준(high-level) 인터페이스
- 산점도(scatterplot)와 회귀선(regplot)은 두 변수 간의 상관관계를 확인하기 좋은 그래프
- 선그래프(lineplot)와 막대그래프(barplot)는 시간의 변화에 따른 추이를 확인하기 좋은 시각화 방법
- 박스플롯(boxplot)과 바이올린플롯(violinplot)은 카테고리별 데이터의 최댓값, 최솟값, 중앙값을 확인할 때 용이
- 히스토그램(histplot)으로 데이터의 분포를 이해할 수 있음
- 히트맵(heatmap)은 여러 변수를 한 번에 비교할 때 유용하게 사용하는 시각화 방법

# seaborn

---

## ● Tips 데이터셋

- seaborn 내장 데이터셋

- 그래프를 그리기 위해서는 데이터가 필요한데, seaborn 패키지에서 제공하는 내장 데이터를 사용하여 실습
- 데이터 수집과 전처리에 소요되는 시간을 줄이고, seaborn 패키지의 다양한 기능을 활용하는 방법을 효율적으로 익힐 수 있도록 정제된 데이터셋 제공

- 데이터 로드 방법

```
import seaborn as sns
```

```
sns.load_dataset(불러올_데이터셋_이름)
```

# seaborn

---

## ● 산점도(scatterplot)

- x축 변수와 y축 변수의 관계를 확인할 때 사용하는 그래프
- 각 변수의 값을 x-y 2차원 평면에 점으로 표시
- 따라서 각각의 점들의 분포 패턴에 따라 상관관계를 판단할 수 있음
- 상관관계의 강도는 점들이 직선에 얼마나 가깝게 분포되어 있는지로 판단
- 점들이 가까울수록 강한 상관관계를 의미
  - 양의 상관관계: 점들이 오른쪽 위로 상승하는 형태
  - 음의 상관관계: 점들이 오른쪽 아래로 하강하는 형태
  - 무상관: 점들이 특별한 패턴없이 분산된 형태
- 기본 사용법

```
import seaborn as sns
```

```
sns.scatterplot(x, y)
```

# seaborn

---

- 회귀선(regplot)

- 회귀란 사전적인 의미로 '한 바퀴 돌아서 본래의 자리나 상태로 돌아오는 것'
- regplot은 개별 데이터 분포를 대표할 수 있는 하나의 선인 회귀(regression)선을 표시하는 그래프
- 이 선은 산점도에 표시된 모든 데이터 포인트들의 전체적인 경향을 하나의 직선으로 요약해서 보여줌

- 기본 사용법

```
import seaborn as sns  
  
sns.regplot(x, y)
```

# seaborn

---

- 선그래프(lineplot)

- 우리 일상 생활에서 자주 접하는 그래프 중 하나
- 일기예보에서 이번주 예상 기온 변화를 나타낸 그래프, 시간에 따른 주가 변화량을 나타내는 그래프 등 주로 시간의 변화에 따른 값의 변화를 나타낼 때 많이 사용
- x축에 연도, 월, 일과 같은 시간을 사용한다면 시간에 따른 데이터 변화 추이를 쉽게 확인 할 수 있음
- 그렇기 때문에 선그래프는 변수의 변화, 트렌드, 변화율 정보가 중요한 경우 사용
- 기본 사용법

```
import seaborn as sns
```

```
sns.lineplot(x, y)
```

# seaborn

---

- 막대그래프(barplot)

- 수치를 막대의 길이로 표현해 시각화하는 방법
- 값들의 차이가 미미하거나 표시할 막대의 수가 많은 경우, 그래프를 이해하기 어려운 문제 발생
- 이러한 경우 선그래프를 활용하거나 막대의 색상을 조절하여 시각화하는 것이 좋음

- 기본 사용법

```
import seaborn as sns
```

```
sns.barplot(x, y)
```



# seaborn

---

- 박스 플롯(boxplot), 바이올린 플롯(violinplot)

- 박스 플롯(boxplot, 상자 그림)은 막대그래프와 유사하게 수치를 길이로 표현하지만, 사분위수를 함께 시각화 함
- 박스 플롯의 각각의 표시선(whisker)은 다음과 같은 의미를 가짐
- 박스 플롯의 구성 요소
  - 박스의 아래선: 1사분위수(Q1, 25%)
  - 박스의 중간선: 중앙값(Q2, 50%)
  - 박스의 위선: 3사분위수(Q3, 75%)
  - 아래쪽 수염:  $Q1 - 1.5 \times IQR$
  - 위쪽 수염:  $Q3 + 1.5 \times IQR$
  - 점들: 이상치(outliers)

# seaborn

---

- 박스 플롯(boxplot) 기본 사용법

```
import seaborn as sns  
  
sns.boxplot(x, y)
```

- 바이올린 플롯(violinplot)

```
import seaborn as sns  
  
sns.violinplot(x, y)
```

# seaborn

---

- 히스토그램(histplot)

- 변수의 분포를 막대형 그래프를 사용하여 표시하는 방법
- x축에 계급을 , y축에 도수를 나타낸 뒤, 각 계급의 크기를 가로의 길이로, 도수를 세로의 길이로 하는 직사각형을 차례로 그려서 표현함
- 데이터의 분포를 이해하는 가장 기본적이면서도 강력한 도구
- 데이터의 정규성 확인, 이상치 탐지, 변수 변환의 필요성 판단 등 탐색적 데이터 분석에서 필수적으로 사용됨
- 기본 사용법

```
import seaborn as sns  
  
sns.histplot(data)
```

# seaborn

---

## ● 히트맵(heatmap)

- 여러 가지 변수를 한 번에 비교할 때 유용하게 사용하는 시각화 방법
- 2차원 격자 모양으로 나뉜 각각의 칸에 데이터의 값을 색으로 표시
- 3차원 데이터(x축, y축, 값)를 2차원 평면에 효과적으로 표현할 수 있는 강력한 도구
- 히트맵의 입력으로 사용되는 데이터의 구조는 기존에 사용했던 1차원 형태가 아닌 2차원 형태의 데이터를 입력 받음
- 히트맵 데이터의 특징
  - x축과 y축: 범주형 데이터(실수형 데이터는 사용하지 않음)
  - 격자 안의 값: 숫자 형태의 변수만 가능
  - 표시할 값: 평균값, 최댓값(max), 합계(sum), 최솟값(min), 개수(count) 등

## • 기본 사용법

```
import seaborn as sns
```

```
sns.heatmap(data)
```

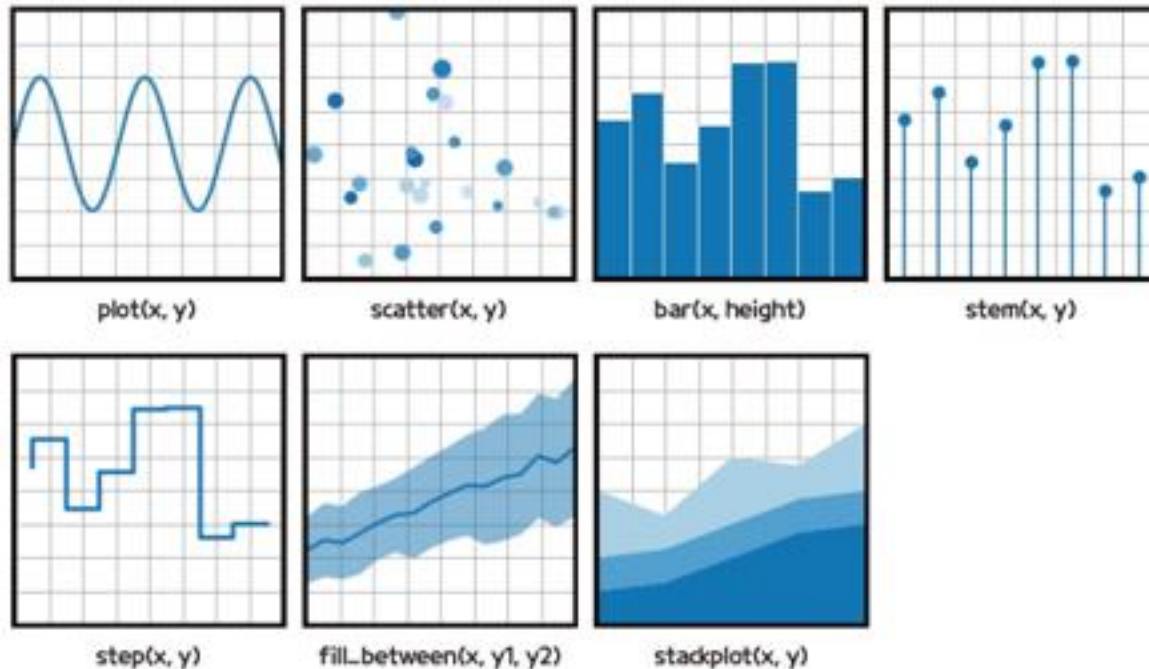
# matplotlib

- 맷플롯립

- 맷플롯립(Matplotlib)은 파이썬 데이터 과학 분야에서 가장 널리 사용되는 시각화 라이브러리

- 그래프 유형

- 기본 플롯: 데이터를  $(x, y)$  쌍으로 표현



# 데이터 시각화 라이브러리

- 그래프 유형

- 배열과 필드 플롯: 데이터  $Z(x, y)$ 와 필드  $U(x, y)$ ,  $V(x, y)$ 의 배열을 표현



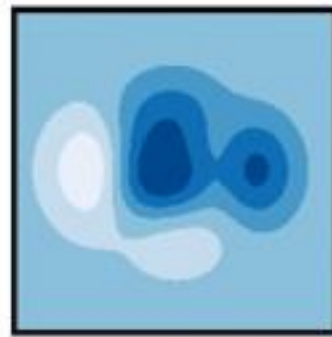
`imshow(Z)`



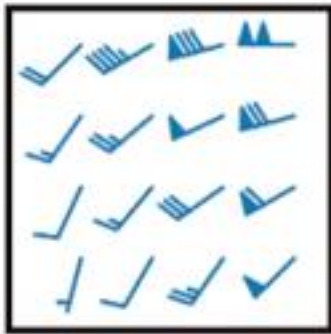
`pcolormesh(X, Y, Z)`



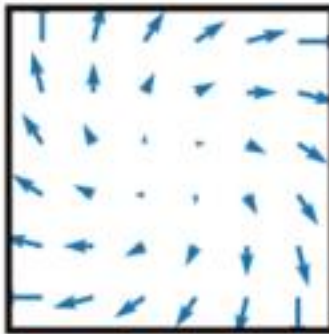
`contour(X, Y, Z)`



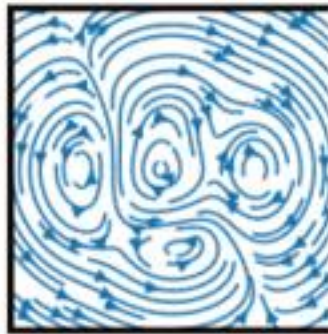
`contourf(X, Y, Z)`



`barbs(X, Y, U, V)`



`quiver(X, Y, U, V)`

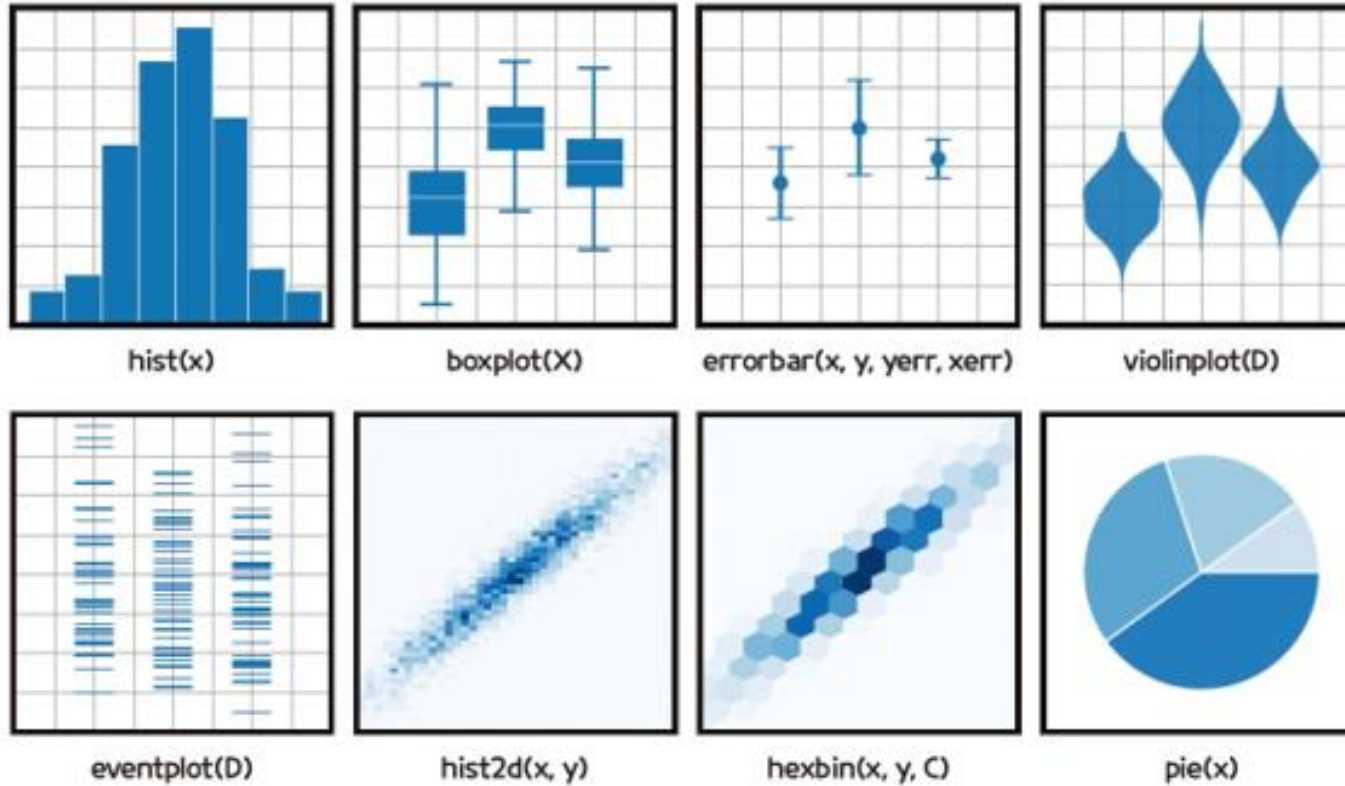


`streamplot(X, Y, U, V)`

# 데이터 시각화 라이브러리

- 그래프 유형

- 통계 플롯: 데이터 분포를 시각화하여 통계 분석에 적합



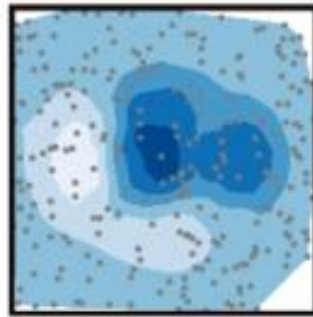
# 데이터 시각화 라이브러리

- 그래프 유형

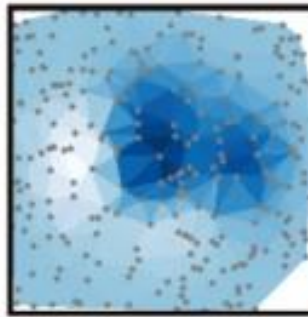
- 비정형 좌표: 좌표( $x, y$ )에서의 값  $z$ 를 등고선으로 시각화



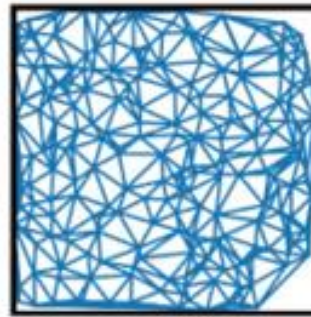
`tricontour(x, y, z)`



`tricontourf(x, y, z)`



`tripcolor(x, y, z)`



`triplot(x, y)`



# 데이터 시각화 라이브러리

- 그래프 유형

- 기본 차트

- 맷플롯립의 하위 패키지인 파이플롯(Pyplot)을 사용
    - 파이플롯의 plot( ) 함수는 꺾은선 그래프를 작성함. 꺾은선 그래프(Line chart)는 시간 데이터를 x축에 두고 연속형 변수의 추세를 살펴볼 때 사용

```
conda install matplotlib
```

```
import matplotlib.pyplot as plt
plt.plot([x값 리스트], y값 리스트, ['포맷 스타일'])
```

포맷 스타일의 종류

|       |                                                               |
|-------|---------------------------------------------------------------|
| 색상    | b(파란색), g(초록색), r(빨간색), c(청록색), y(노란색), k(검은색), w(흰색)         |
| 선 종류  | - (실선), -- (파선), -. (일점쇄선), : (점선)                            |
| 마커 모양 | o (원), + (+ 기호), D (다이아몬드), s (사각형), ^ (삼각형), v (역삼각형), . (점) |

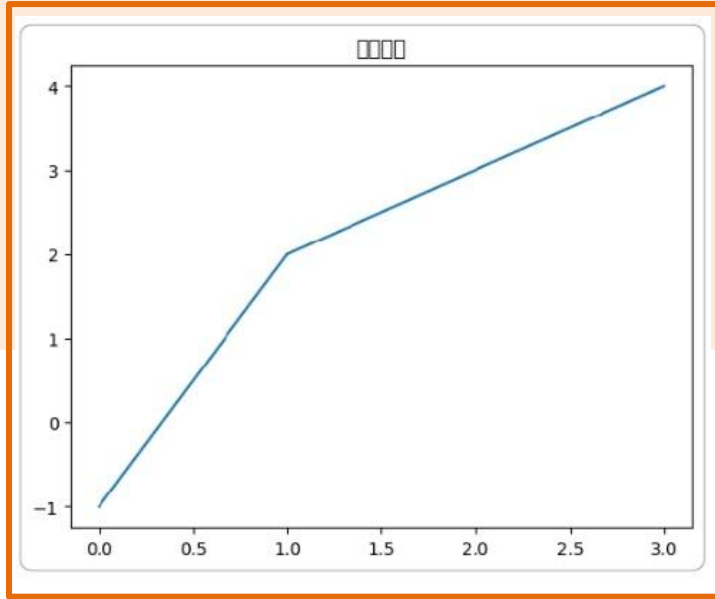
# 데이터 시각화 라이브러리

- 그래프 유형
  - 기본 차트

## 차트 생성

```
import matplotlib.pyplot as plt

plt.title('대한민국')
plt.plot([-1, 2, 3, 4])
plt.show()
```



- 파이플롯의 show() 함수를 마지막에 적으면 속성 관련 메시지는 생략되고 그래프만 깔끔하게 출력

# 데이터 시각화 라이브러리

- 그래프 유형

- 기본 차트

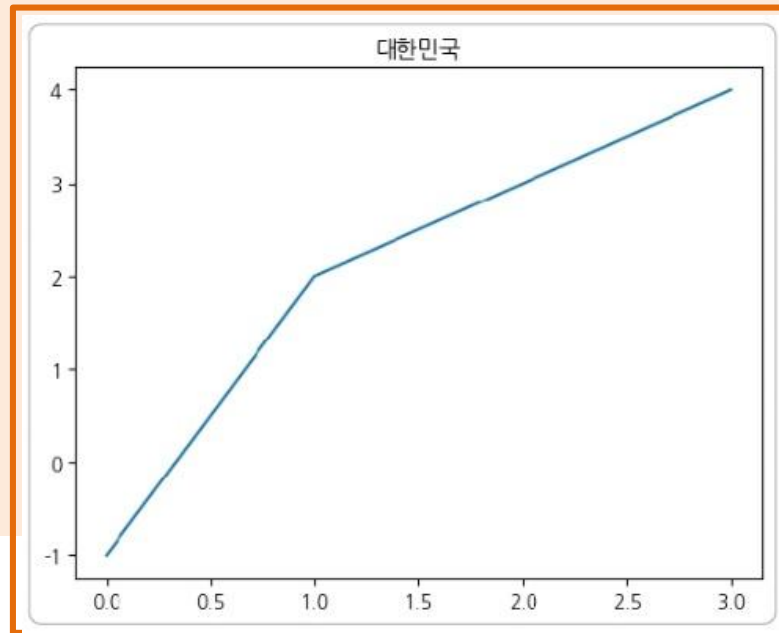
- 같은 그래프를 다시 출력하여 폰트가 적용되었는지 확인

## 한글 폰트 설치

```
import matplotlib.pyplot as plt

plt.rc('font', family='NanumGothic')
plt.rcParams['axes.unicode_minus'] = False

plt.plot([-1, 2, 3, 4])
plt.title('대한민국')
plt.show()
```



# 데이터 시각화 라이브러리

- 그래프 유형

- 기본 차트

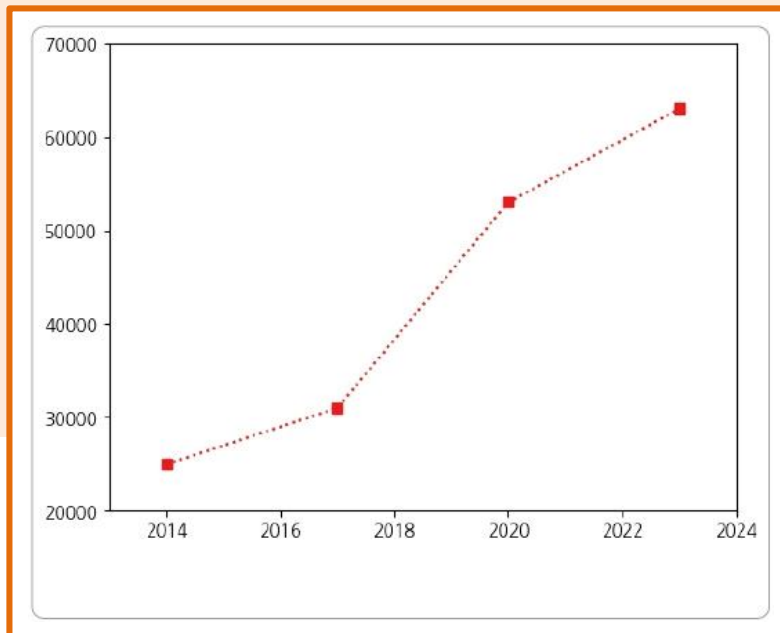
- 맷플롯립 파이플롯의 `axis()` 함수로 축 범위를 지정할 수 있음

```
plt.axis(x 최솟값, x 최댓값, y 최솟값, y 최댓값)
```

## 축 범위를 지정하여 차트 생성

```
year = [2014, 2017, 2020, 2023]
price = [25000, 31000, 53000, 63000]
plt.plot(year, price, 'rs:')

plt.axis([2013, 2024, 20000, 70000])
plt.show()
```



# 데이터 시각화 라이브러리

## ● 그래프 유형

### • 기본 차트

- x축을 공동으로 사용해 한 차트에 그래프 여러 개를 나타낼 수 있음

#### 차트 하나에 그래프 세 개 생성

```
import numpy as np
```

```
plt.plot(np.random.randn(10), 'k', label='one')
```

#검은색 실선

```
plt.plot(np.random.randn(10)*3, 'r--', label='two')
```

#빨간색 파선

```
plt.plot(np.random.randn(10)*10, 'g.', label='three')
```

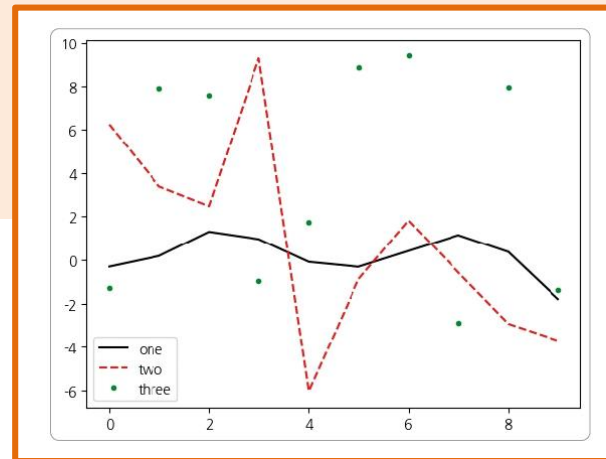
#선 없이 초록색 점

마커로 표시

```
plt.legend()
```

```
plt.show()
```

- legend() 함수로 범례를 표시



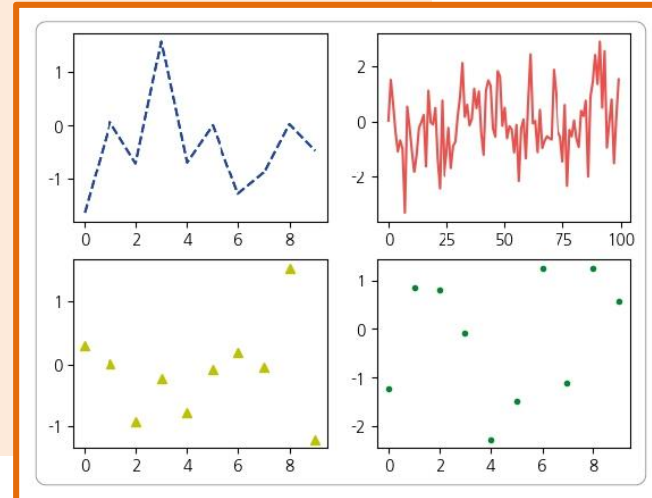
# 데이터 시각화 라이브러리

## ● 그래프 유형

- 다중 그래프
  - 다중 그래프(Multi graph)는 축이 여러 개
  - 인덱스는 파이썬의 기본적인 인덱싱과 조금 다르니 주의

### 다중 그래프

```
plt.subplot(2, 2, 1)          # 2행 2열 중 첫 번째 그래프
plt.plot(np.random.randn(10), 'b--')
plt.subplot(2, 2, 2)          # 2행 2열 중 두 번째 그래프
plt.plot(np.random.randn(100), 'r', alpha=0.7)
plt.subplot(2, 2, 3)          # 2행 2열 중 세 번째 그래프
plt.plot(np.random.randn(10), 'y^')
plt.subplot(2, 2, 4)          # 2행 2열 중 네 번째 그래프
plt.plot(np.random.randn(10), 'g.')
plt.show()
```



# 데이터 시각화 라이브러리

- 그래프 유형

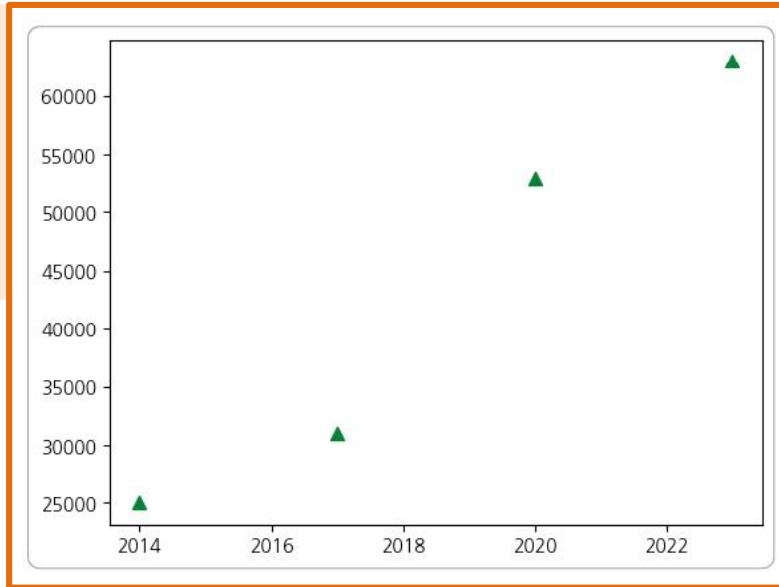
- 산점도

- 산점도(Scatter Plot)로 두 연속형 변수 값의 분포 또는 상관관계를 나타낼 수 있음

```
plt.scatter(x값 리스트, y값 리스트, c='색상', s=크기, marker='모양')
```

## 연도별 상품가격 산점도

```
year = [2014, 2017, 2020, 2023]
price = [25000, 31000, 53000, 63000]
plt.scatter(year, price, c='g', s=50, marker='^')
plt.show()
```



# 데이터 시각화 라이브러리

- 그래프 유형

- 히스토그램

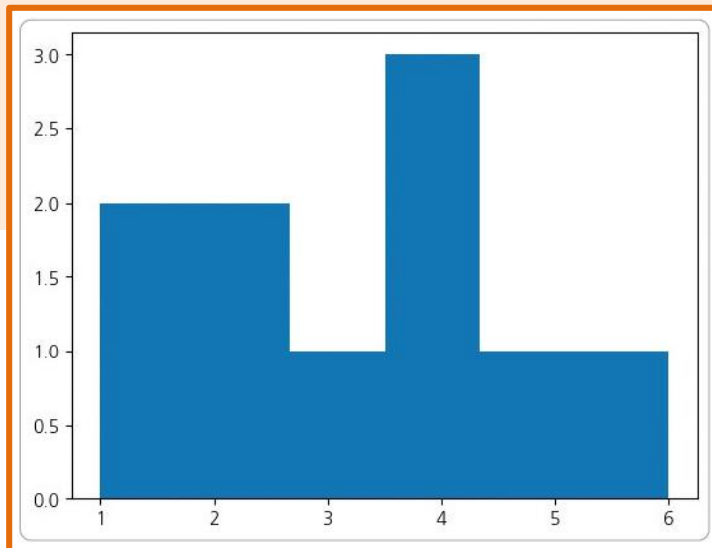
- 히스토그램(Histogram)으로 연속형 변수의 구간별 분포를 나타낼 수 있음

```
plt.hist(값 리스트, bins=구간의 수)
```

- 주사위를 10번 던졌을 때 1부터 6까지의 수가 몇 번씩 나왔는지 히스토그램으로

## 주사위 결과 히스토그램

```
numbers = [5, 4, 4, 1, 6, 3, 4, 1, 2, 2]
plt.hist(numbers, bins=6)
plt.show()
```



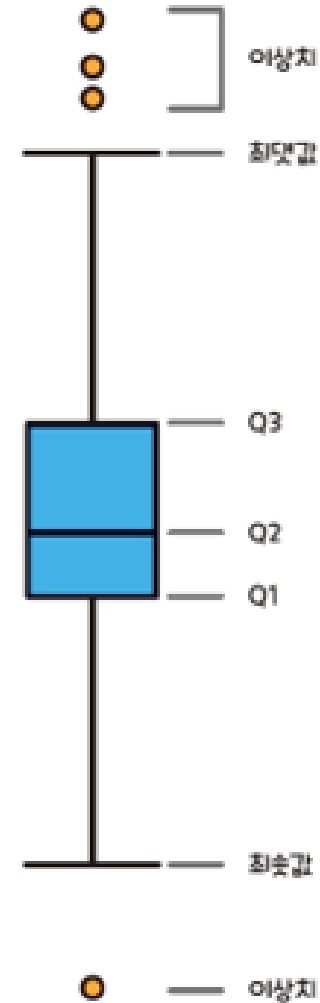


# 데이터 시각화 라이브러리

## ● 그래프 유형

### • 상자 그래프

- 상자 그래프(Box plot)로 연속형 변수의 분포를 살펴볼 수 있음
  - 상자 그래프에 최댓값, 최솟값, 중앙값, 사분위수를 표시
- ① 제1사분위수(Q1)와 제3사분위수(Q3)를 양 끝으로 하는 상자를 그림
  - ② 상자 안에 중앙값을 수평선으로 표시
  - ③ 상자의 길이, 즉 Q1에서 Q3까지의 범위를 사분범위(IQR, Inter-quartile Range)라고 하며 이 상자 길이의 1.5배 안에 있는 데이터를 기준으로 수염을 표시
  - ④ 수염 바깥에 있는 데이터는 작은 원으로 표현하며, 이상치(Outliers)로 간주

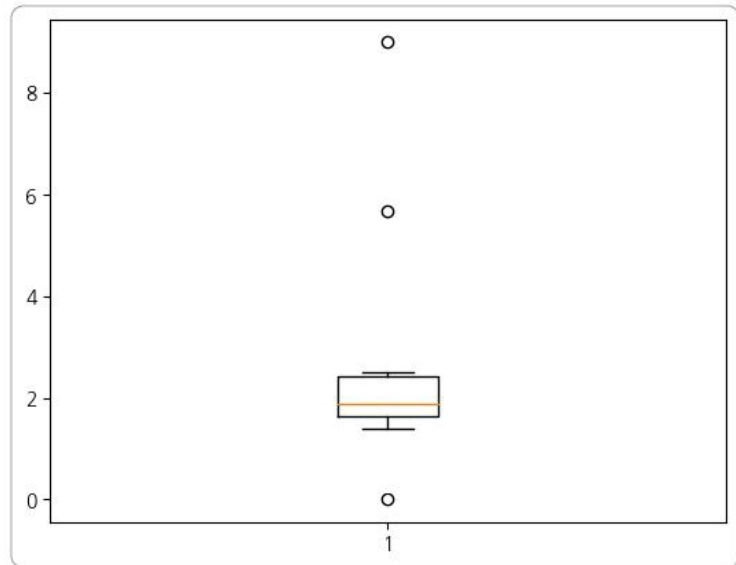


# 데이터 시각화 라이브러리

- 그래프 유형
  - 상자 그래프

## 상자 그래프

```
numbers = [0, 1.4, 1.6, 1.8, 1.85, 1.9, 2.2, 2.5, 5.7, 9]  
plt.boxplot(numbers)  
plt.show()
```



# 데이터 시각화 라이브러리

- 그래프 유형

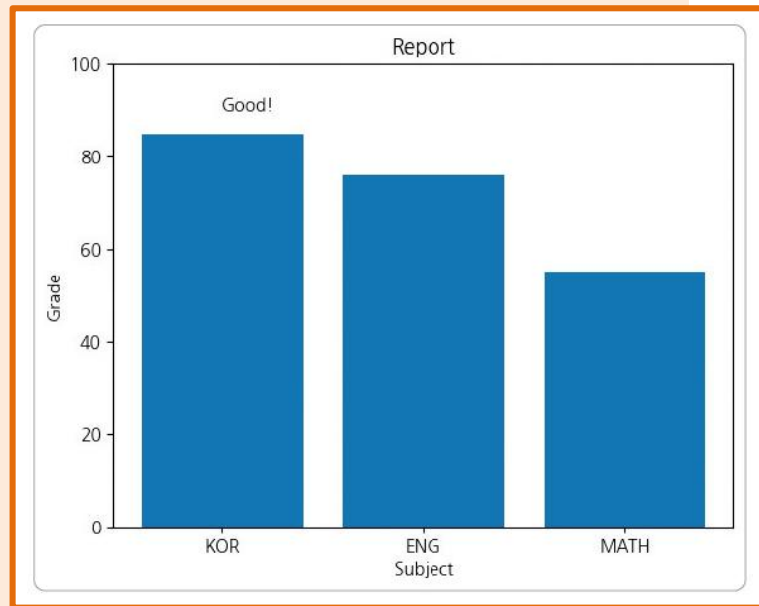
- 막대 그래프

- 막대 그래프(Bar plot)로 범주형 변수끼리 연속형 값을 비교할 수 있음

```
plt.bar(x값 리스트, y값 리스트)
```

## 과목별 성적 막대 그래프

```
subject = ['KOR', 'ENG', 'MATH']  
grade = [85, 76, 55]  
  
plt.title('Report')  
plt.xlabel('Subject')  
plt.ylabel('Grade')  
  
plt.bar(subject, grade)  
plt.ylim(0,100)           #y축 범위  
plt.text(0,90, 'Good!')  
plt.show()
```



# 데이터 시각화 라이브러리

## ● 그래프 유형

### • 파이 차트

- 파이 차트(Pie chart)로 범주형 변수끼리 연속형 값의 비율을 비교할 수 있음

```
plt.pie(값 리스트, labels=레이블 리스트, autopct='%.숫자f%%', colors=색상, explode=띄우는 값 리스트)
```

- 소비량 사과 20, 바나나 15, 체리 5, 키위 5, 포도가 10인 파이 차트를 작성

### 과일 소비량 파이 차트

```
import pandas as pd
example_series = pd.Series([20, 15, 5, 5, 10],\
                           index=['Apple', 'Banana', 'Cherry', 'Kiwi', 'Grape'])
plt.pie(example_series, labels=example_series.index, autopct='%.1f%%')
plt.show()
```

